

Documentation

★ How BRAIN works

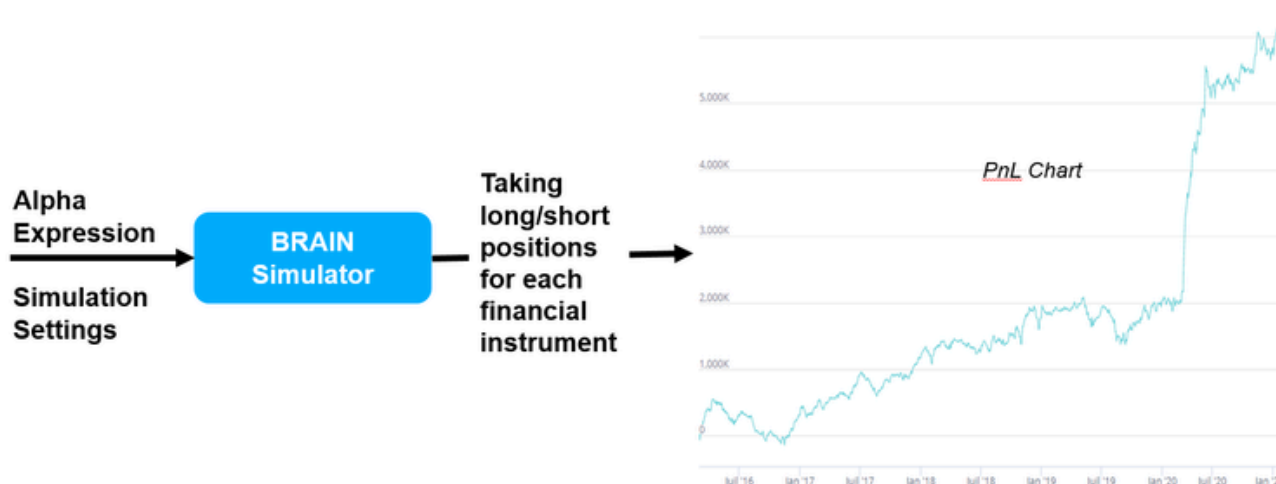
The below post illustrates in detail how the BRAIN platform works and what happens in the background when you simulate an Alpha. Even though you'll never need to do these calculations yourselves, developing an intuition for them will help you in the Alpha making process.

Imagine market data being a matrix, with each row representing one date and each column representing one stock. For example, the matrix for close price data of stocks in [universe US TOP3000](#) would look like this:

Dates	MSFT	HOG	AAPL	GOOG	PG	...
20100104	30.95	25.46	214.01	626.75	61.12	...
20100105	30.96	25.65	214.38	623.99	61.14	...
20100106	30.77	25.59	210.97	608.26	60.85	...
20100107	30.452	25.8	210.58	594.1	60.52	...
20100108	30.66	25.53	211.98	602.02	60.44	...

When you input the simulation settings and click “Simulate”, the BRAIN platform will evaluate the Alpha expression against the matrix of market data for each date in a five year span, taking a long or short position for each financial instrument to generate the PnL chart.

When you click “Simulate”:



generate:

Normally, in an Alpha simulation, there would be between 200 and 3,000 stock instruments in the universe. But to better understand this concept, we'll assume a hypothetical scenario in which the simulation universe has only eight stocks. We simulate the expression **rank(-returns)** with market neutralization, Delay 1 and Decay 0 settings for now.

The hypothesis in this expression is that we want to buy, or go long on, those stocks tomorrow that had negative or comparatively lower returns today, and we want to sell, or go short on, those stocks tomorrow that had positive or comparatively higher returns today.

We've used the rank operator, which ranks the input values inside the operator and return values equally distributed between zero and 1. This is an example of a reversion idea.

	B	C	D	E	F	G	H	I	J	K	L
1	How does the Expression become a PnL curve										
2	Stocks in the Alpha vector	Stock Returns on 1-Feb	Alpha value on 2-Feb : rank(-returns)	Subtract Average	Centred around 0	Absolute value	Normalized weights on 2-Feb	Absolute value	Allocate \$ 20 Mn capital (Position on 2nd Feb)	Suppose Returns on 2 Feb are	\$Mn profits on 2 Feb would be
3	Stock 1	3.0%	0.00	(0.50)	-0.5	0.50	-0.22	0.22	-4.4	2.0%	-0.09
4	Stock 2	2.5%	0.14	(0.50)	-0.4	0.36	-0.16	0.16	-3.1	2.0%	-0.06
5	Stock 3	1.8%	0.29	(0.50)	-0.2	0.21	-0.09	0.09	-1.9	-2.1%	0.04
6	Stock 4	0.6%	0.43	(0.50)	-0.1	0.07	-0.03	0.03	-0.6	-2.1%	0.01
7	Stock 5	0.2%	0.57	(0.50)	0.1	0.07	0.03	0.03	0.6	1.0%	0.01
8	Stock 6	-0.9%	0.71	(0.50)	0.2	0.21	0.09	0.09	1.9	0.0%	0.00
9	Stock 7	-1.7%	0.86	(0.50)	0.4	0.36	0.16	0.16	3.1	1.0%	0.03
10	Stock 8	-2.8%	1.00	(0.50)	0.5	0.50	0.22	0.22	4.4	2.0%	0.09
11	Sum				-1.1 + 1.1	2.3	-0.5 + 0.5	1.0	+10 M -10 M		0.03
12	Mean		0.50		0.0		0.0	Normalized	0.0		P&L

In Column B, we have the eight stocks in the Alpha vector. Column C shows the returns of these stocks as of February 1st. These serve as the input data of the Alpha expression.

Step1: Evaluate the expression for each stock to generate the Alpha vector for the given date.

In our case, this date would be February 2nd, because we've assumed Delay 1 settings. The Delay 1 setting uses data as of T-1 date to create the Alpha vector as of T date.

To produce the Alpha vector, the simulator performs the rank operation on negative returns and produces a vector of values corresponding to each stock.

Simulate

Alphas

Learn

Data

Competitions (3)

	Stocks in the Alpha vector	Stock Returns on 1-Feb	Alpha value on 2-Feb : rank(-returns)	Subtract Average	Centred around 0	Absolute value	Normalized weights on 2-Feb	Absolute value	Allocate \$ 20 Mn capital (Position on 2nd Feb)	Suppose Returns on 2 Feb are	\$Mn profits on 2 Feb would be
3	Stock 1	3.0%	0.00	(0.50)	-0.5	0.50	-0.22	0.22	-4.4	2.0%	-0.09
4	Stock 2	2.5%	0.14	(0.50)	-0.4	0.36	-0.16	0.16	-3.1	2.0%	-0.06
5	Stock 3	1.8%	0.29	(0.50)	-0.2	0.21	-0.09	0.09	-1.9	-2.1%	0.04
6	Stock 4	0.6%	0.43	(0.50)	-0.1	0.07	-0.03	0.03	-0.6	-2.1%	0.01
7	Stock 5	0.2%	0.57	(0.50)	0.1	0.07	0.03	0.03	0.6	1.0%	0.01
8	Stock 6	-0.9%	0.71	(0.50)	0.2	0.21	0.09	0.09	1.9	0.0%	0.00
9	Stock 7	-1.7%	0.86	(0.50)	0.4	0.36	0.16	0.16	3.1	1.0%	0.03
10	Stock 8	-2.8%	1.00	(0.50)	0.5	0.50	0.22	0.22	4.4	2.0%	0.09
11	Sum				-1.1 + 1.1	2.3	-0.5 + 0.5	1.0	+10 M -10 M		0.03
12	Mean		0.50		0.0		0.0	Normalized	0.0		P&L

The resulting vector depends on the operators used in the Alpha expression. In our case, since we've used the rank operator, we see equally distributed values between 0 and 1 in Column D. Note that the stock with the lowest return has the highest value, and vice versa, in line with our hypothesis.

Step 2: From each value in the vector, subtract the average of the vector values in the group. Sum of all vector values = 0. This is called neutralization.

The group can be the entire market, but we can also perform this neutralization operation on sector, industry or subindustry groupings of stocks.

	B	C	D	E	F	G	H	I	J	K	L
1	How does the Expression become a PnL curve										
2	Stocks in the Alpha vector	Stock Returns on 1-Feb	Alpha value on 2-Feb : rank(-returns)	Subtract Average	Centred around 0	Absolute value	Normalized weights on 2-Feb	Absolute value	Allocate \$ 20 Mn capital (Position on 2nd Feb)	Suppose Returns on 2 Feb are	\$Mn profits on 2 Feb would be
3	Stock 1	3.0%	0.00	(0.50)	-0.5	0.50	-0.22	0.22	-4.4	2.0%	-0.09
4	Stock 2	2.5%	0.14	(0.50)	-0.4	0.36	-0.16	0.16	-3.1	2.0%	-0.06
5	Stock 3	1.8%	0.29	(0.50)	-0.2	0.21	-0.09	0.09	-1.9	-2.1%	0.04
6	Stock 4	0.6%	0.43	(0.50)	-0.1	0.07	-0.03	0.03	-0.6	-2.1%	0.01
7	Stock 5	0.2%	0.57	(0.50)	0.1	0.07	0.03	0.03	0.6	1.0%	0.01
8	Stock 6	-0.9%	0.71	(0.50)	0.2	0.21	0.09	0.09	1.9	0.0%	0.00
9	Stock 7	-1.7%	0.86	(0.50)	0.4	0.36	0.16	0.16	3.1	1.0%	0.03
10	Stock 8	-2.8%	1.00	(0.50)	0.5	0.50	0.22	0.22	4.4	2.0%	0.09
11	Sum				-1.1 + 1.1	2.3	-0.5 + 0.5	1.0	+10 M -10 M		0.03
12	Mean		0.50		0.0		0.0	Normalized	0.0		P&L

Since we have only eight stocks in our simulation universe, we've assumed to neutralize the stocks over the market.

So we take the average of the numbers in Cell D12 and subtract the average from each stock. This gives us a new vector in Column F. Note that both the sum and the average of these new numbers are now zero. Also, the sum of positive values is equal to the sum of negative values.

Step 3: The resulting values are scaled or 'normalized' such that absolute sum of the Alpha vector values is 1. These values can be called as normalized weights.

Simulate

Alphas

Learn

Data

Competitions (3)

	Stocks in the Alpha vector	Stock Returns on 1-Feb	Alpha value on 2-Feb : rank(-returns)	Subtract Average	Centred around 0	Absolute value	Normalized weights on 2-Feb	Absolute value	Allocate \$ 20 Mn capital (Position on 2nd Feb)	Suppose Returns on 2 Feb are	\$Mn profits on 2 Feb would be
3	Stock 1	3.0%	0.00	(0.50)	-0.5	0.50	-0.22	0.22	-4.4	2.0%	-0.09
4	Stock 2	2.5%	0.14	(0.50)	-0.4	0.36	-0.16	0.16	-3.1	2.0%	-0.06
5	Stock 3	1.8%	0.29	(0.50)	-0.2	0.21	-0.09	0.09	-1.9	-2.1%	0.04
6	Stock 4	0.6%	0.43	(0.50)	-0.1	0.07	-0.03	0.03	-0.6	-2.1%	0.01
7	Stock 5	0.2%	0.57	(0.50)	0.1	0.07	0.03	0.03	0.6	1.0%	0.01
8	Stock 6	-0.9%	0.71	(0.50)	0.2	0.21	0.09	0.09	1.9	0.0%	0.00
9	Stock 7	-1.7%	0.86	(0.50)	0.4	0.36	0.16	0.16	3.1	1.0%	0.03
10	Stock 8	-2.8%	1.00	(0.50)	0.5	0.50	0.22	0.22	4.4	2.0%	0.09
11	Sum				-1.1 + 1.1	2.3	-0.5 + 0.5	1.0	+10 M -10 M		0.03
12	Mean		0.50		0.0		0.0	Normalized	0.0		P&L

That means, we sum the absolute values of each row and find the sum, which is 2.3. Then we divide each row by this sum, which results in normalized values. By normalize, we mean that the total absolute sum of Column H is 1. We can also call this vector a normalized vector of weights.

Note: On each iteration/day, the expression rank(-returns) will have access to all the data for returns up to that day, and the matrix will grow by one line every day until it reaches the most recent date. The role of the expression is to transform the input matrix to an output vector of weights as we see in this hypothetical example.

Step 4: Using normalized weights, the BRAIN simulator allocates capital (from a fictitious book of \$20 million) to each stock to construct a portfolio.

	B	C	D	E	F	G	H	I	J	K	L
1	How does the Expression become a PnL curve										
2	Stocks in the Alpha vector	Stock Returns on 1-Feb	Alpha value on 2-Feb : rank(-returns)	Subtract Average	Centred around 0	Absolute value	Normalized weights on 2-Feb	Absolute value	Allocate \$ 20 Mn capital (Position on 2nd Feb)	Suppose Returns on 2 Feb are	\$Mn profits on 2 Feb would be
3	Stock 1	3.0%	0.00	(0.50)	-0.5	0.50	-0.22	0.22	-4.4	2.0%	-0.09
4	Stock 2	2.5%	0.14	(0.50)	-0.4	0.36	-0.16	0.16	-3.1	2.0%	-0.06
5	Stock 3	1.8%	0.29	(0.50)	-0.2	0.21	-0.09	0.09	-1.9	-2.1%	0.04
6	Stock 4	0.6%	0.43	(0.50)	-0.1	0.07	-0.03	0.03	-0.6	-2.1%	0.01
7	Stock 5	0.2%	0.57	(0.50)	0.1	0.07	0.03	0.03	0.6	1.0%	0.01
8	Stock 6	-0.9%	0.71	(0.50)	0.2	0.21	0.09	0.09	1.9	0.0%	0.00
9	Stock 7	-1.7%	0.86	(0.50)	0.4	0.36	0.16	0.16	3.1	1.0%	0.03
10	Stock 8	-2.8%	1.00	(0.50)	0.5	0.50	0.22	0.22	4.4	2.0%	0.09
11	Sum				-1.1 + 1.1	2.3	-0.5 + 0.5	1.0	+10 M -10 M		0.03
12	Mean		0.50		0.0		0.0	Normalized	0.0		P&L

Column J has a total of \$20 million of fictional money allocated to the stocks, using the normalized weights in Column H. This means we have a position of minus \$4.4 million in Stock 1 — that is, we've shorted \$4.4 million worth of Stock 1 — and a long position of \$0.6 million in Stock 5. That is, we've invested \$0.6 million in Stock 5.

This is called long-short market neutralization, and it's the backbone of creating these predictive models, or Alphas, on BRAIN. With this technique, a strategy can have the potential to be profitable regardless of the direction of the market.

Step 5: Calculate next day PnL generated by the Alpha based on observed stock returns the next day

Each stock, based on the returns each stock had that day.

	B	C	D	E	F	G	H	I	J	K	L
1	How does the Expression become a PnL curve										
2	Stocks in the Alpha vector	Stock Returns on 1-Feb	Alpha value on 2-Feb : rank(-returns)	Subtract Average	Centred around 0	Absolute value	Normalized weights on 2-Feb	Absolute value	Allocate \$ 20 Mn capital (Position on 2nd Feb)	Suppose Returns on 2 Feb are	\$Mn profits on 2 Feb would be
3	Stock 1	3.0%	0.00	(0.50)	-0.5	0.50	-0.22	0.22	-4.4	2.0%	-0.09
4	Stock 2	2.5%	0.14	(0.50)	-0.4	0.36	-0.16	0.16	-3.1	2.0%	-0.06
5	Stock 3	1.8%	0.29	(0.50)	-0.2	0.21	-0.09	0.09	-1.9	-2.1%	0.04
6	Stock 4	0.6%	0.43	(0.50)	-0.1	0.07	-0.03	0.03	-0.6	-2.1%	0.01
7	Stock 5	0.2%	0.57	(0.50)	0.1	0.07	0.03	0.03	0.6	1.0%	0.01
8	Stock 6	-0.9%	0.71	(0.50)	0.2	0.21	0.09	0.09	1.9	0.0%	0.00
9	Stock 7	-1.7%	0.86	(0.50)	0.4	0.36	0.16	0.16	3.1	1.0%	0.03
10	Stock 8	-2.8%	1.00	(0.50)	0.5	0.50	0.22	0.22	4.4	2.0%	0.09
11	Sum				-1.1 + 1.1	2.3	-0.5 + 0.5	1.0	+10 M -10 M		0.03
12	Mean		0.50		0.0		0.0	Normalized	0.0		P&L

Suppose the actual returns on these stocks as of February 2nd are as shown in Column K. We see that although we expected Stock 1 and Stock 2 to fall in price, they actually went up, so we had a loss, shown in Column L.

We expected Stock 6 to go up in price, but it stayed flat. So we were wrong about three stocks, but we were right about five. In total, we made a gain of \$0.03 million on this day with our Alpha, calculated by adding the PnLs of all stocks in our vector. This is how the simulator calculates the PnL generated by the Alpha for any given date.

Step 6: Perform the operations in Step 1 to Step 5 for each date in a several-year history span also called the *In-sample period (IS)* to get daily PnL generated for each day

For each day, the expression is evaluated and the values in the Alpha output vector represent the weights to allocate to each stock. Alpha weights are not how much you want to buy or sell, but a weighting position you would reach this day. These weights are multiplied by book size (total money invested in the portfolio) to get the dollar value held in each stock. For example, if the Alpha weight (after [neutralization](#) and scaling) for MSFT is 0.2423, then we'll have MSFT stocks with the total value 0.2423*book size.

The [weight](#) can be negative, meaning you would take a short position on these stocks. If the value is positive, you would take a long position on these stocks, i.e. buy the stocks. A [NAN](#) value would mean no weight is allocated to that instrument (i.e. no money is allocated). The value of stocks you buy/sell on a particular day is determined by the difference between Alpha weights today and weights yesterday. The percentage of your portfolio traded in a day (by dollar value) is called 'turnover'. The [turnover](#) reported in [simulation](#) results is the average daily turnover over the simulation.

Step 7: Calculate the cumulative PnL of the Alpha from the start of the in-sample period to get the PnL chart of the Alpha.

Based on those daily positions, [PnL](#) is calculated and displayed. By default, the BRAIN

connection of the Alpha to the market.

This can be better understood with the help of the PnL chart of the Alpha in our example rank(-returns)

Chart

PnL



In this chart, we have an IS period of five years, from February 2016 to January 2021. Using the steps from the example, the simulator would calculate the daily PnL of the Alpha and derive the cumulative PnL chart, as we see here. Note that the two years from February 2021 to January 2023 are not visible to us in the simulation window. That's called the out-of-sample, or the OS, period. After you submit an Alpha, several tests are run to analyze the Alpha's performance in the OS period. An Alpha that passes both the in-sample and out-of-sample tests can be said to be a robust Alpha.

This is how the BRAIN simulator creates the PnL chart from an Alpha.

In our example, we've assumed that we're using market neutralization and Decay 0

The simulator would perform the same operations (from step 1 to step 5) on each of the ten groups and finally add the PnL from each group to get the daily PnL of the Alpha and create the cumulative PnL chart (Step 6 and Step 7)

However, if we introduce decay into our Alpha settings, an additional step must be performed to get the final Alpha vector.

Suppose we use a decay of 3 in our simulation settings. The final vector of weights in the Alpha would be calculated by combining today's value with the previous day's decayed value. In our example, we calculated the normalized weights in the Alpha as of February 2nd. Let's assume that the normalized weights of stocks in the Alpha vector on February 1st and January 31st are as shown in Columns N and O, respectively.

	B	H	I	J	K	L	M	N	O	P
1	How does the							Decay = 3		
2	Stocks in the Alpha vector	Normalized weights on 2-Feb	Absolute value	Allocate \$ 20 Mn capital (Position on 2nd Feb)	Suppose Returns on 2 Feb are	\$Mn profits on 2 Feb would be		Normalized weights on 1-Feb	Normalized weights on 31-Jan	Final weight after Decay
3	Stock 1	-0.22	0.22	-4.4	2.0%	-0.09		-0.03	0.03	-0.11
4	Stock 2	-0.16	0.16	-3.1	2.0%	-0.06		-0.16	-0.03	-0.14
5	Stock 3	-0.09	0.09	-1.9	-2.1%	0.04		-0.22	0.09	-0.11
6	Stock 4	-0.03	0.03	-0.6	-2.1%	0.01		0.03	-0.16	-0.03
7	Stock 5	0.03	0.03	0.6	1.0%	0.01		-0.09	0.22	0.02
8	Stock 6	0.09	0.09	1.9	0.0%	0.00		0.22	-0.22	0.08
9	Stock 7	0.16	0.16	3.1	1.0%	0.03		0.09	0.16	0.13
10	Stock 8	0.22	0.22	4.4	2.0%	0.09		0.16	-0.09	0.15
11	Sum	-0.5 + 0.5	1.0	+10 M -10 M		0.03				
12	Mean	0.0	Normalized	0.0		P&L				

Then the final weights in the Alpha would be calculated using the given weighted average formula:

$$Decay_linear(x, n) = \frac{x[date] * n + x[date - 1] * (n - 1) + \dots + x[date - n + 1]}{n + (n - 1) + \dots + 1}$$

which is implemented in Column P. Using this new derived vector, the simulator would calculate the daily PnL and consequently the cumulative PnL chart. Note that even if decay is used, more weight is assigned to the most recent values. So decay is an important factor in reducing transaction costs or turnover, as it includes information from previous days, preventing the Alpha from being reactive.

To summarize, once we input the Alpha expression and simulation settings in the BRAIN simulator, it performs the operations discussed above to take long or short positions for each financial instrument and generates the PnL chart.

[Prev: How to choose the Simulation Settings](#)

